Информационные системы управления данными и документами

ФИО ст.преподавателя: Тесла-Блозва Святослав Александрович

e-mail: teslablozva@mirea.ru

Что такое Vaadin?

Полнофункциональный фреймворк для создания Enterprise Web UI на Java.

- Vaadin — Java-фреймворк с открытым исходным кодом.
- Позволяет писать веб-интерфейсы полностью на Java без написания HTML/JavaScript/CSS.
- Создает и управляет деревом веб-компонентов непосредственно на JVM сервера



Vaadin.com

Архитектура: Client-Side vs Server-Side

Две парадигмы веб-разработки

Характеристика	SPA (React / Angular / Vue)	Vaadin (Server-Side Approach)
Место выполнения UI-логики	Браузер (Client JavaScript)	JVM на сервере (Server Java)
Связь с сервером	REST API / GraphQL (DTO)	WebSocket / HTTP (Flow Engine Sync)
Состояние (State)	Хранится в браузере (Redux/Pinia)	Хранится в сессии на сервере
Требования к стеку	Java + JavaScript/TypeScript	Только Java

Преимущества Server-Side подхода

Почему Server-Side актуален в Enterprise?

- **Единый стек -** разработчику нужен только Java (меньше требований к знанию JS-экосистемы).
- **Прямой доступ к backend -** нет необходимости создавать сотни DTO и REST-контроллеров для форм.
- **Безопасность -** бизнес-логика и правила валидации не передаются клиенту в виде JS-кода.
- **Скорость разработки (Time-to-Market) -** повышение производительности при создании сложных таблиц и форм.

Ограничения Server-Side подход

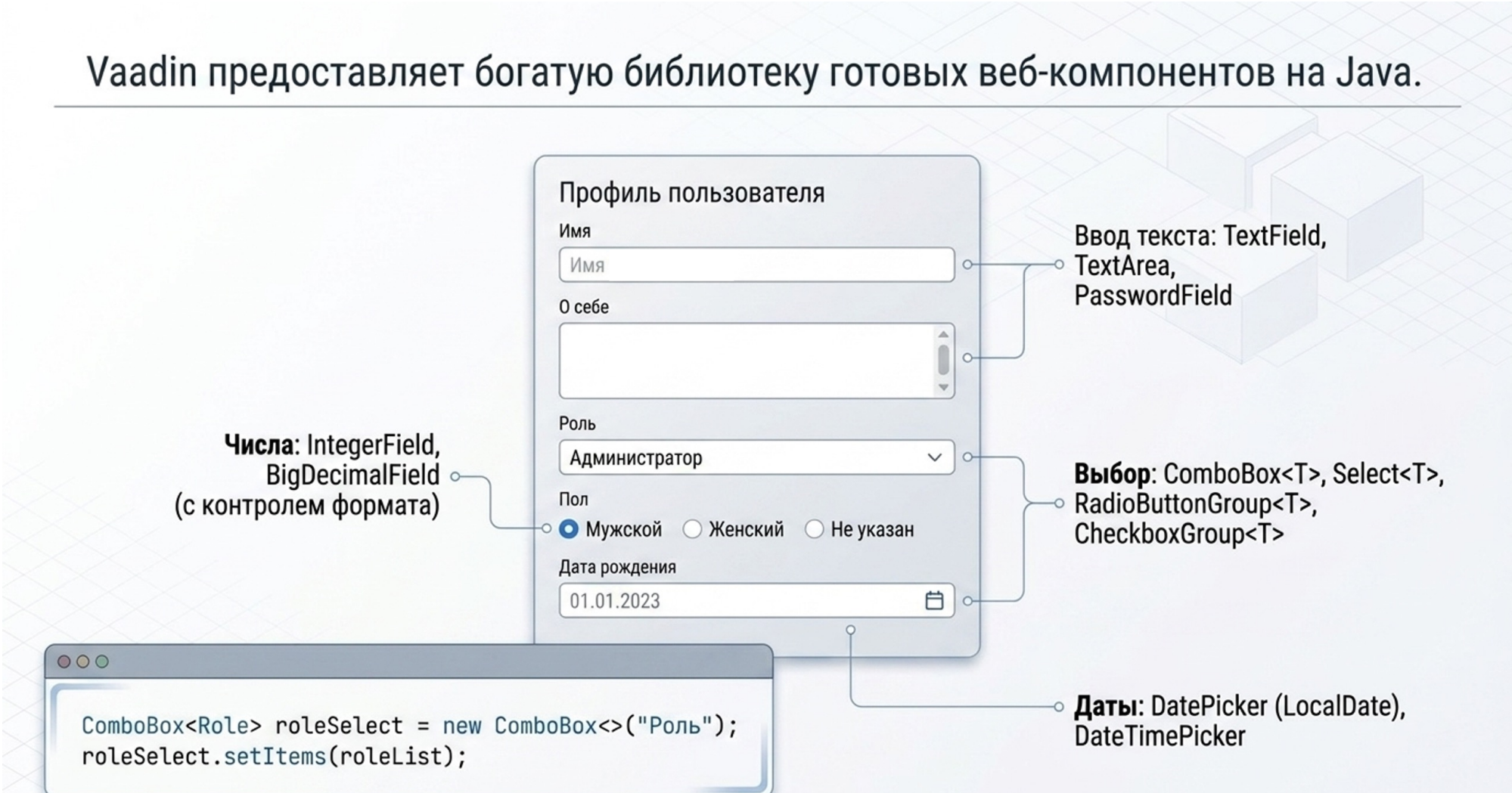
Когда Server-Side требует внимания?

- **Расходование памяти -** сервер хранит дерево UI для каждой активной сессии.
- **Зависимость от сети -** любое действие (клик, ввод) требует отправки запроса на сервер.
- **Решение -** профилирование сессий, масштабирование серверов приложений, грамотная очистка неиспользуемых компонентов.

Как Vaadin работает под капотом?



Компоненты Vaadin



Иерархия компонентов Vaadin

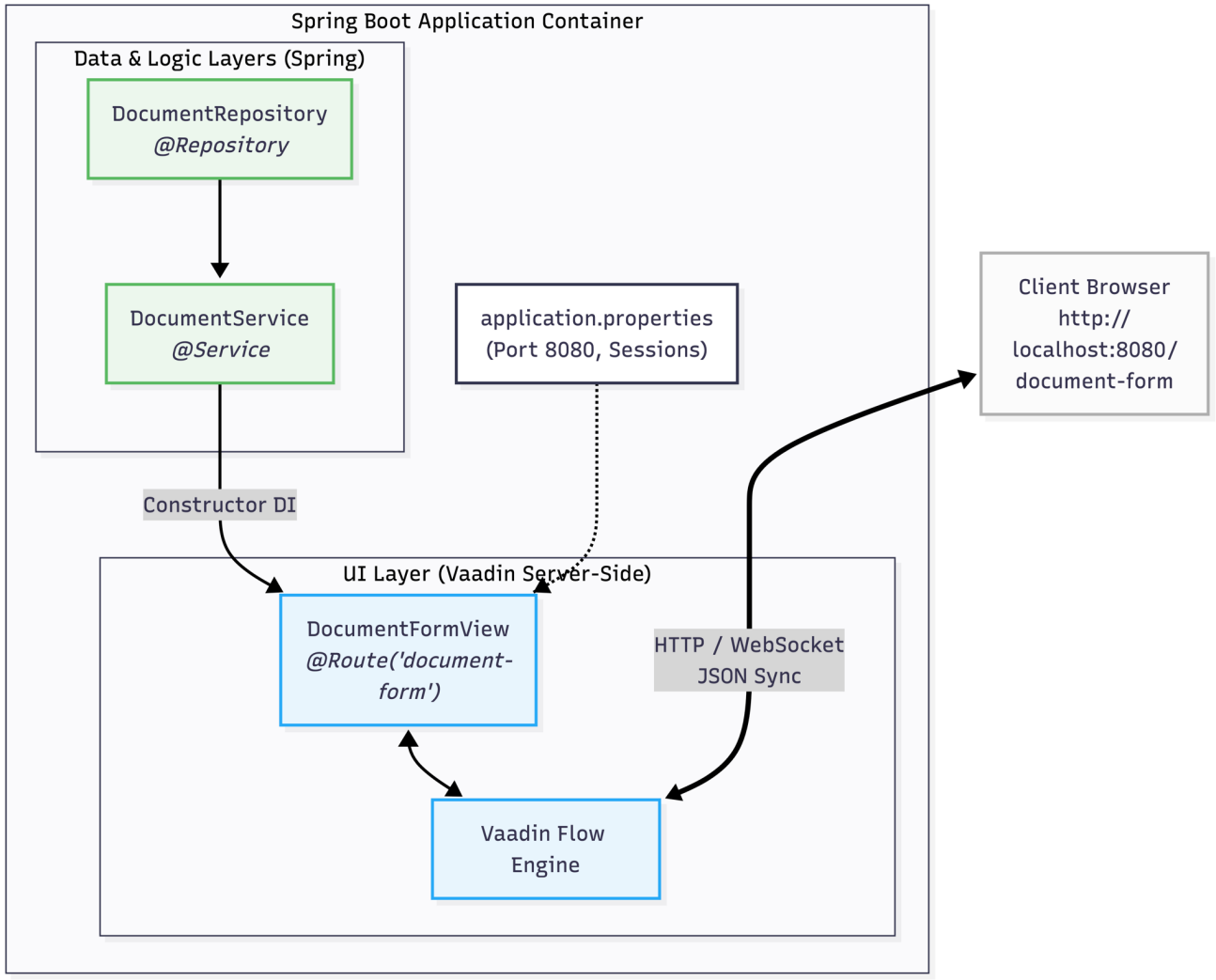
- com.vaadin.flow.component.Component — абстрактный корень всех компонентов UI.
- HasValue — интерфейс для компонентов с изменяемым значением (TextField, Checkbox, ComboBox).
- HasText / HasComponents — контейнеры и текстовые элементы.
- HasSize / HasStyle — управление размерами и CSS-стилями

Интеграция Vaadin с фреймворком Spring Boot

В современном корпоративном стеке Vaadin запускается как часть приложения Spring Boot. Это дает разработчику ряд преимуществ:

- **Автоматическое связывание.** Все UI-компоненты, помеченные аннотацией @Route, автоматически становятся веб-страницами и Spring-бинами.
- **Внедрение зависимостей (DI).** В UI-представления можно напрямую инжектировать сервисы бизнес-логики и репозитории через конструктор.
- **Единая конфигурация.** Управление портом приложения, параметрами сессий и базами данных происходит в едином файле application.properties.

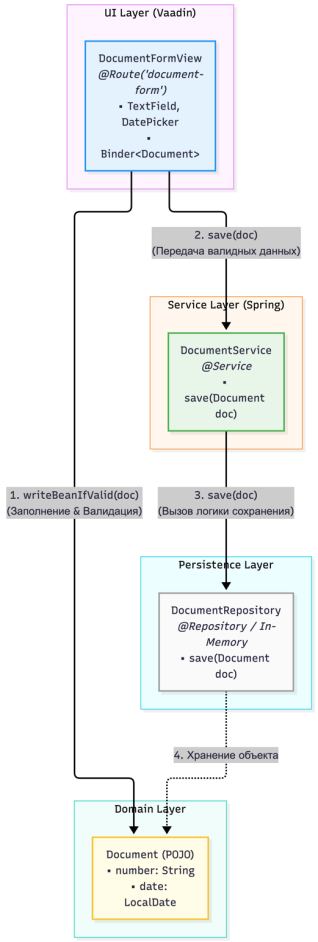
Интеграция Vaadin с фреймворком Spring Boot



Интеграция Vaadin с фреймворком Spring Boot

- **Единый контейнер Spring Boot**
  Всё приложение работает в одном JVM-процессе на сервере — фронтенд и бэкенд не нужно развёртывать отдельно. Параметры окружения (порт, HTTP-настройки, сессии) задаются централизованно в application.properties.
- **Слой данных и бизнес-логики**
  - DocumentRepository (@Repository) — отвечает за операции с базой данных (чтение, запись, обновление).
  - DocumentService (@Service) — содержит бизнес-логику, обрабатывает запросы от UI и вызывает репозиторий.
- **Серверный UI на Vaadin**
  В отличие от SPA (React/Angular), вся логика интерфейса выполняется на сервере:
  - DocumentFormView (@Route) — класс формы, автоматически становящийся веб-страницей по заданному URL.
  - Зависимости (например, сервис) внедряются напрямую через конструктор — без REST-контроллеров и DTO.
  - Vaadin Flow синхронизирует состояние серверных компонентов с браузером, храня дерево UI в памяти сессии.
- **Клиент и синхронизация**
  Браузер — «тонкий клиент»: только отрисовывает DOM. Любое действие пользователя (клик, ввод) отправляется на сервер в JSON (через HTTP/WebSocket). Сервер обрабатывает событие, обновляет состояние и автоматически возвращает изменения в браузер.

Архитектура модуля: Entity – Service – View



Архитектура модуля: Entity – Service – View

При создании пользовательских форм в корпоративных приложениях рекомендуется следовать разделению обязанностей:

- **Domain Model (Document.java).** Обычный Java-класс (POJO) с геттерами и сеттерами, хранящий состояние данных.
- **Service Layer (DocumentService.java).** Компонент Spring (@Service), отвечающий за сохранение, обработку и валидацию данных на стороне бэкенда.
- **UI Layer (DocumentFormView.java).** Компонент Vaadin (@Route), строящий экранную форму, собирающий ввод пользователя через Binder и передающий объект в сервис.

Вводные компоненты (Text & Input)

Текстовые поля и ввод данных

- TextField — однострочное текстовое поле.
- TextArea — многострочный ввод текста.
- PasswordField — поле ввода пароля с маскированием.
- IntegerField / BigDecimalField — численные поля с контролем формата

Пример - создание простых полей

```
TextField titleField = new TextField("Заголовок документа");
titleField.setPlaceholder("Введите название...");
titleField.setClearButtonVisible(true);
titleField.setMaxLength(100);

IntegerField priorityField = new IntegerField("Приоритет");
priorityField.setValue(1);
priorityField.setHasControls(true); // кнопки + / -
```

Выборные компоненты (Selection UI)

Выбор элементов из списка

- ComboBox<T> — выпадающий список с автодополнением и поддержкой фильтрации.
- Select<T> — классический выпадающий HTML-список.
- RadioButtonGroup<T> — группа радио-кнопок для одиночного выбора.
- CheckboxGroup<T> — группа чекбоксов для множественного выбора

Пример - использование ComboBox

Работа с ComboBox и объектами

```
ComboBox<Department> deptCombo = new ComboBox<>("Отдел");
deptCombo.setItems(departmentService.findAll());
deptCombo.setItemLabelGenerator(Department::getName);
deptCombo.addValueChangeListener(event -> {
    Department selected = event.getValue();
    Notification.show("Выбран отдел: " + selected.getName());
});
```

Компоненты дат и времени

Выбор дат и времени

- DatePicker — выбор даты из календаря (работает с LocalDate).
- TimePicker — выбор времени (работает с LocalTime).
- DateTimePicker — комбинированный выбор даты и времени (LocalDateTime).

Локализация - настройка формата дат и наименований дней/месяцев.

Макеты компоновки (Layouts)

Принципы размещения элементов (Layouts)

Компоненты в Vaadin размещаются внутри специальной контейнерной структуры.

- VerticalLayout — элементы располагаются сверху вниз.
- HorizontalLayout — элементы располагаются слева направо.
- FlexLayout — гибкая верстка на основе CSS Flexbox.
- FormLayout — специализированный макет для адаптивных бизнес-форм.

VerticalLayout и HorizontalLayout

Линейная компоновка

```
HorizontalLayout headerBar = new HorizontalLayout();
headerBar.setWidthFull();
headerBar.setAlignItems(Alignment.CENTER);

Button saveBtn = new Button("Сохранить");
Button cancelBtn = new Button("Отмена");

headerBar.add(new Label("Редактирование"), saveBtn, cancelBtn);
headerBar.expand(saveBtn); // Занимает всё свободное место
```

FormLayout и его особенности

Макет бизнес-форм: FormLayout

Автоматически адаптирует количество колонок в зависимости от ширины экрана.

- По умолчанию использует 2 колонки на ПК и 1 колонку на мобильных устройствах.
- Позволяет настраивать свойство colspan для объединения колонок.
- Автоматически выравнивает подписи (labels) к полям.

Пример - FormLayout на практике

Код настройки FormLayout

```
FormLayout form = new FormLayout();
TextField fn = new TextField("Имя");
TextField ln = new TextField("Фамилия");
EmailField email = new EmailField("Email");

form.add(fn, ln, email);
// Объединяем email на 2 колонки
form.setColspan(email, 2);

form.setResponsiveSteps(
    new FormLayout.ResponsiveStep("0", 1),
    new FormLayout.ResponsiveStep("500px", 2)
);
```

Отображение списков - Grid

Компонент Grid - Табличные данные

Основной инструмент для вывода реестров объектов в Атом.Платформе.

- Поддерживает ленивую загрузку данных (Lazy Loading / Paging).
- Позволяет настраивать сортировку, фильтрацию и кастомные рендеры колонок.
- Встроенная поддержка выбора строк (Single / Multi Select).

Пример - настройка Grid

Код создания и настройки Grid

```
Grid<RaContract> grid = new Grid<>(RaContract.class, false);

grid.addColumn(RaContract::getNumber).setHeader("Номер");
grid.addColumn(RaContract::getSum).setHeader("Сумма");
grid.addColumn(c -> c.getCompany().getName()).setHeader("Контрагент");

grid.addComponentColumn(contract -> {
    Button edit = new Button("Редактировать");
    edit.addClickListener(e -> openEditor(contract));
    return edit;
});
grid.setItems(contractService.findAll());
```

Переход к связыванию данных

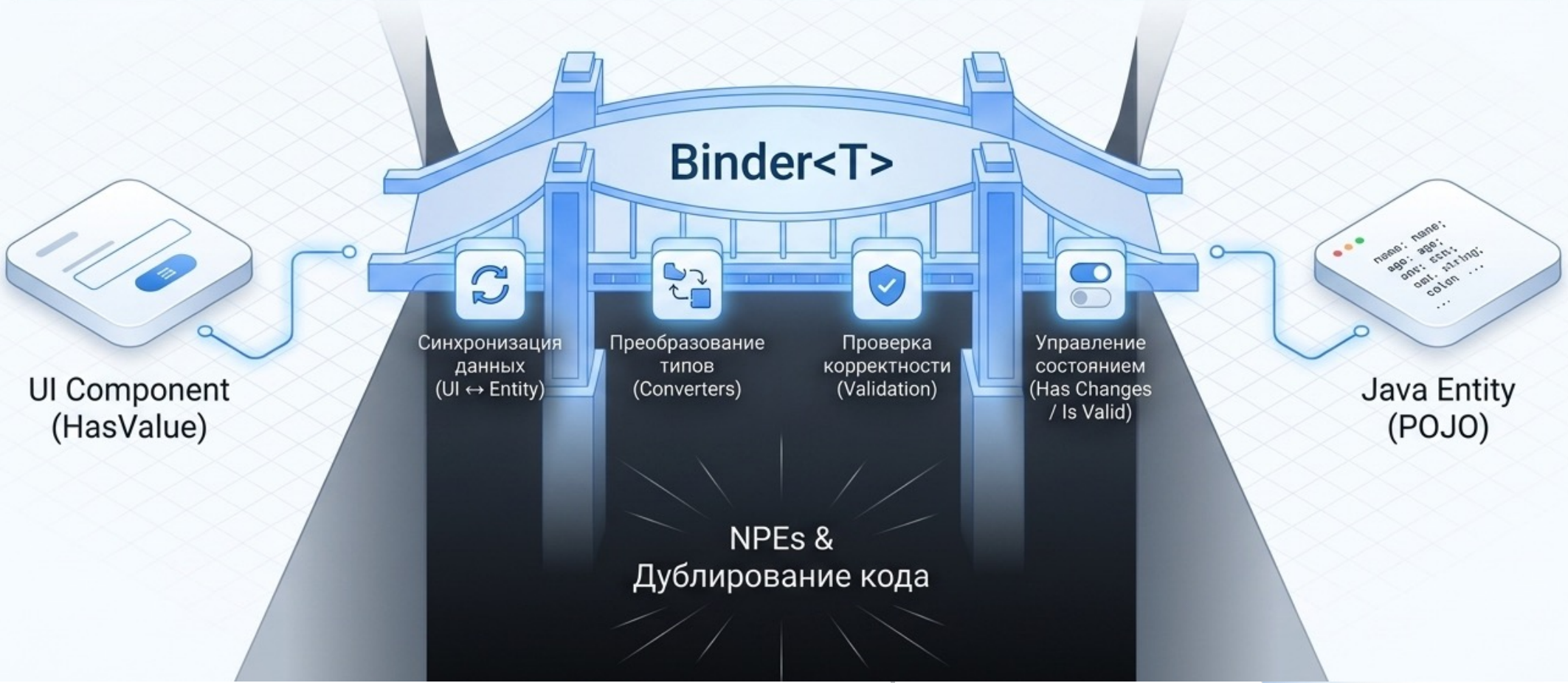
Мост между UI и доменной моделью

Как перенести данные из Java-объекта (POJO) в поля формы и обратно?

Проблема ручного связывания - ручные присвоения приводят к дублированию кода, отсутствию валидации и NPE.

Решение Vaadin - использование механизма Binder<T>.

Binder — это интеллектуальный мост между UI-компонентамкл и доменной моделью (POJO).



Класс Binder - Концепция

Что такое Binder<T>?

Binder<T> — это связующее звено между классом сущности T и компонентами UI (HasValue).

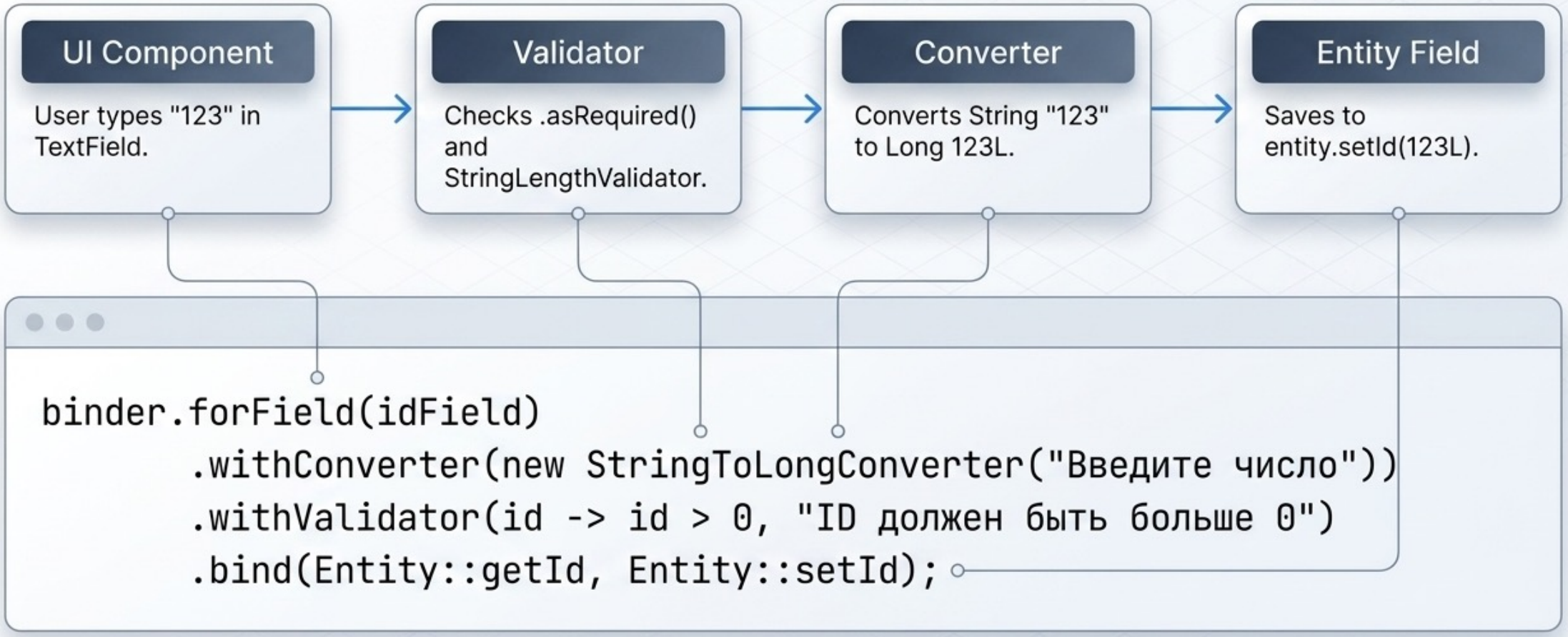
- Синхронизация данных в обе стороны (UI ↔ Entity).
- Преобразование типов (Converters).
- Проверка корректности введенных данных (Validation).
- Управление состоянием формы (Has Changes / Is Valid).

Архитектура Binder

Внутренняя структура связывания

- UI Component (TextField) ↔ Validator ↔ Converter ↔ Entity Field (String/Long)
- Для каждого поля создается свой BindingBuilder.
- Данные проходят цепочку проверок перед тем, как попасть в объект

Данные проходят строгую цепочку проверок до изменения доменной модели.



Простейший пример использования Binder

Базовый сценарий Binder

```
Binder<RaContract> binder = new Binder<>(RaContract.class);

TextField numberField = new TextField("Номер договора");

binder.forField(numberField)
    .bind(RaContract::getNumber, RaContract::setNumber);

RaContract contract = new RaContract();
binder.readBean(contract); // Загрузка данных в UI
```

Чтение и запись данных: readBean vs writeBean

Методы управления данными в Binder

- binder.readBean(entity) — заносит значения из entity в UI. Изменения полей не меняют entity автоматически.
- binder.writeBean(entity) — валидирует UI и сохраняет значения в entity. Возвращает ValidationException, если есть ошибки.
- binder.setBean(entity) — устанавливает **двустороннее** автосвязывание (любое изменение в UI сразу меняет entity).

Автоматическое связывание по именам

Автоматический биндинг: bindInstanceFields

Сокращение кода при совпадении имён полей UI и свойств сущности.

```
public class ContractForm extends FormLayout {
    private TextField number; // Имя совпадает с RaContract.number
    private BigDecimalField sum; // Имя совпадает с RaContract.sum

    private Binder<RaContract> binder = new Binder<>(RaContract.class);

    public ContractForm() {
        add(number, sum);
        binder.bindInstanceFields(this); // Автоматический поиск
    }
}
```

Валидация данных в Binder

Валидация - Гарантия корректности данных

Валидаторы проверяют пользовательский ввод до изменения сущности.

- .asRequired("Поле обязательно")
- .withValidator(new StringLengthValidator(...))
- .withValidator(new EmailValidator(...))
- .withValidator(new RegexpValidator(...))

Пример добавления валидаторов

Цепочка валидации

```
binder.forField(titleField)
    .asRequired("Название не может быть пустым")
    .withValidator(
        title -> title.length() >= 5,
        "Длина должна быть не менее 5 символов"
    )
    .bind(RaContract::getTitle, RaContract::setTitle);
```

Кастомные валидаторы

Создание пользовательской логики валидации

```
binder.forField(contractDateField)
    .asRequired("Укажите дату")
    .withValidator(date -> !date.isBefore(LocalDate.now()),
        "Дата договора не может быть в прошлом!")
    .bind(RaContract::getDate, RaContract::setDate);
```

Преобразование типов (Converters)

Конвертеры данных (Converters)

Нужны, когда тип поля UI отличается от типа свойства сущности.

```
binder.forField(stringCodeField)
    .withConverter(
        new StringToLongConverter("Введите корректное число"))
    .bind(RaContract::getCode, RaContract::setCode);
```

Комплексный пример. Binder с конвертером и валидатором

Полная цепочка обработки

```
binder.forField(priceField)
    .asRequired("Укажите цену")
    .withConverter(new StringToBigDecimalConverter("Неверный
формат числа"))
    .withValidator(price -> price.compareTo(BigDecimal.ZERO) > 0,
                   "Цена должна быть больше нуля")
    .bind(RaContract::getPrice, RaContract::setPrice);
```

Событийная модель (Event-Driven UI)

Обработка событий в Vaadin

UI реагирует на действия пользователя через слушатели (Listeners).

- ClickEvent — клик по кнопке или элементу.
- ValueChangeEvent — изменение значения в поле ввода.
- AttachEvent / DetachEvent — добавление или удаление компонента из DOM

Сравнение современных парадигм веб-интерфейсов

Характеристика	Event-Driven UI (Vaadin / Blazor)	Reactive UI (React / Vue / Angular)	Request-Driven (Spring MVC + Thymeleaf)
Где хранится State	На сервере в сессии JVM	В браузере (Redux, Pinia, Local State)	Отсутствует (Stateless / DB)
Модель обновления	Императивные слушатели событий	Декларативная (`UI = f(State)`)	Полная перезагрузка страницы
Передача данных	JSON-дельта по WebSocket / XHR	REST API / GraphQL (DTO)	Готовый HTML по HTTP GET/POST
Сложность разработки	Низкая (только Java-код)	Высокая (два стека: JS + Java)	Минимальная (классический монолит)

Пример - обработка событий клика и изменения

Код регистрации событий

```
Button saveBtn = new Button("Сохранить");
saveBtn.addClickListener(event -> saveContract());

// Горячая клавиша Enter
saveBtn.addClickShortcut(Key.ENTER);

TextField searchField = new TextField();
searchField.setValueChangeMode(ValueChangeMode.LAZY);
searchField.setValueChangeTimeout(500); // Debounce 500ms
searchField.addValueChangeListener(e -> updateList(e.getValue()));
```

Оптимизация событийного потока: Debounce & Throttle

Проблема - при динамическом поиске или вводе текста пользователь генерирует десятки событий нажатия клавиш в секунду. Отправка каждого события на сервер перегрузит сеть и базу данных.

Решения:

Debounce (Задержка паузы)

Throttle (Ограничение частоты)

Обратная связь - Notification

Уведомления пользователя (Notification)

Короткие всплывающие сообщения для информирования о статусе операций.

```
Notification notification = Notification.show(
    "Документ успешно сохранен!",
    3000,
    Notification.Position.BOTTOM_END
);
notification.addThemeVariants(NotificationVariant.LUMO_SUCCESS);
```

Диалоговые окна (Dialog)

Модальные окна и подтверждения

Dialog блокирует основной UI для получения решения от пользователя.

```
Dialog dialog = new Dialog();
dialog.setHeaderTitle("Удаление договора");

VerticalLayout content = new VerticalLayout(new Text("Вы уверены?"));
dialog.add(content);

Button confirm = new Button("Удалить", e -> {
    deleteContract();
    dialog.close();
});
confirm.addThemeVariants(ButtonVariant.LUMO_PRIMARY, ButtonVariant.LUMO_ERROR);

dialog.getFooter().add(confirm, new Button("Отмена", e -> dialog.close()));
dialog.open();
```

Стилизация и темы (Lumo Theme)

Внешний вид компонентов. Встроенная тема Lumo

- Vaadin использует современную тему дизайна **Lumo** по умолчанию.
- Поддерживает светлый (Light) и тёмный (Dark) варианты.
- Настройка цвета и размеров производится через CSS-переменные (--lumo-primary-color).
- Компоненты имеют варианты оформления (Variants): LUMO_PRIMARY, LUMO_TERTIARY, LUMO_SMALL.

Пример стилизации компонентов

Применение вариантов Lumo

```
Button primaryBtn = new Button("Главное действие");
primaryBtn.addThemeVariants(ButtonVariant.LUMO_PRIMARY);

Button dangerBtn = new Button("Опасное действие");
dangerBtn.addThemeVariants(ButtonVariant.LUMO_ERROR);

TextField smallField = new TextField("Компактное поле");
smallField.addThemeVariants(TextFieldVariant.LUMO_SMALL);
```

Концепция Server-Driven UI (SDUI) в индустрии

Что такое SDUI?

Подход, при котором сервер управляет не только данными, но и структурой, логикой и отображением пользовательского интерфейса.

Аналоги в других экосистемах:

- **Microsoft Blazor Server (.NET) -**Прямой аналог Vaadin в мире C#. Выполняет C#-код на сервере и синхронизирует DOM через SignalR (WebSocket).
- **Phoenix LiveView (Elixir) & Laravel Livewire (PHP) -**Серверные фреймворки, отправляющие минимальные HTML-диффы по WebSocket при изменении состояния.
- **SDUI в мобильной разработке (iOS / Android) -**Подход, используемый в Яндексе, Avito, Ozon, где бэкенд присылает мобильному клиенту JSON-схему экрана, позволяя изменять интерфейс без обновления приложения через App Store / Google Play.

Создание форм в Атом.Платформе

Специфика UI в Атом.Платформе

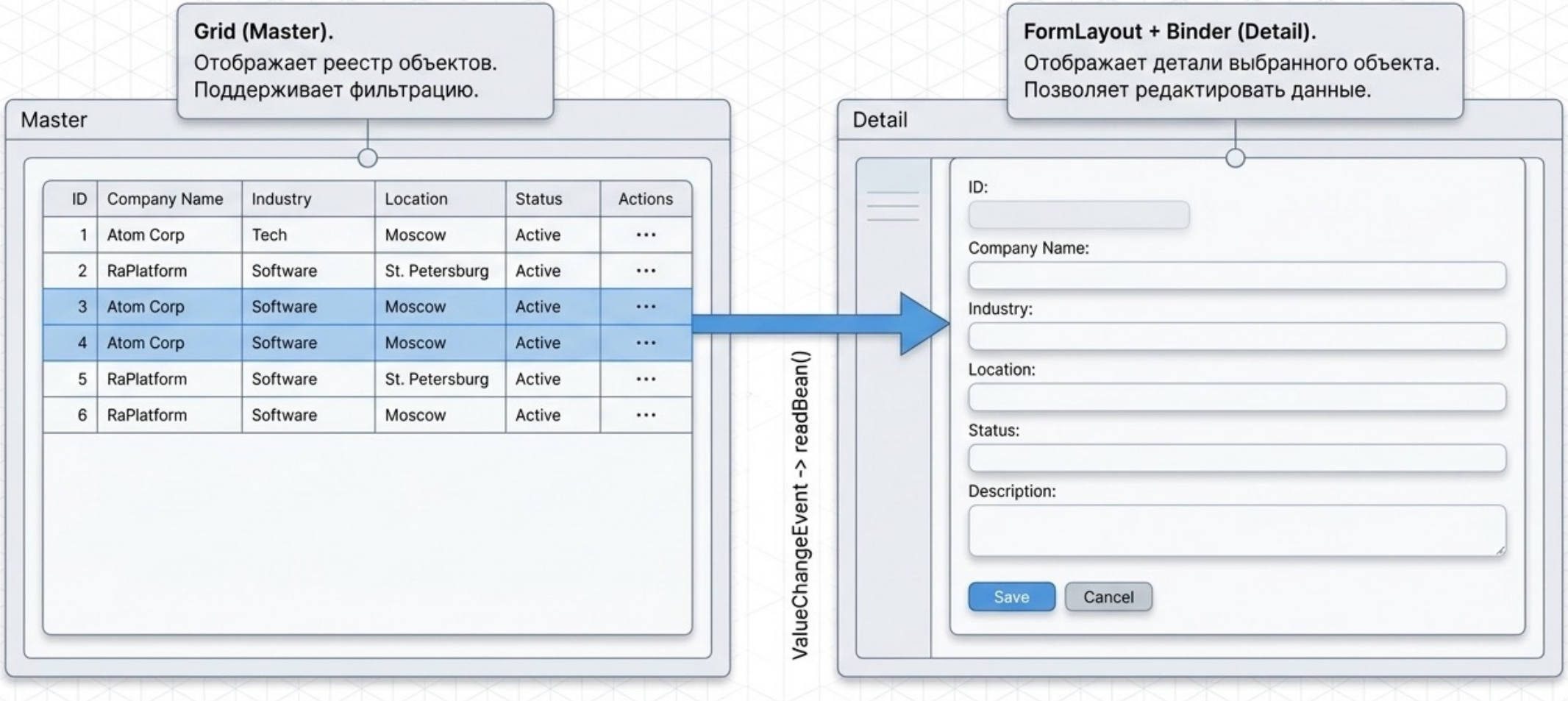
- Базовые объектные типы платформы (например, RaContract, RaCompany) отображаются через унифицированные формы.
- Сочетание правил доступа (ACL/ABAC из Лекции 1) с визуальной доступностью полей UI (field.setReadOnly(!hasWriteAccess)).
- Использование сервисного слоя платформы для сохранения сущностей.

Архитектурный паттерн Master-Detail

Паттерн «Master-Detail» для реестров

Компонент	Назначение в интерфейсе
Grid (Master)	Отображает список/реестр объектов, поддерживает фильтрацию и быстрый выбор записи.
FormLayout (Detail)	Отображает детали выбранного объекта, позволяет редактировать или создавать новые данные.

Паттерн Master-Detail — стандарт де-факто для корпоративных реестров Атом.Платформы.



Проектирование класса формы (Шаг 1)

Объявление класса формы

```
public class RaContractForm extends CustomComponent {
    private TextField number = new TextField("Номер");
    private DatePicker signDate = new DatePicker("Дата подписания");
    private ComboBox<RaCompany> company = new
ComboBox<>("Контрагент");

    private Button save = new Button("Сохранить");
    private Button cancel = new Button("Отмена");

    private Binder<RaContract> binder = new Binder<>(RaContract.class);
}
```

Инициализация UI и макета (Шаг 2)

Сборка макета формы

```
public RaContractForm(List<RaCompany> companies) {
    company.setItems(companies);
    company.setItemLabelGenerator(RaCompany::getName);

    FormLayout formLayout = new FormLayout();
    formLayout.add(number, signDate, company);

    HorizontalLayout buttons = new HorizontalLayout(save, cancel);

    VerticalLayout mainLayout = new VerticalLayout(formLayout, buttons);
    setCompositionRoot(mainLayout);

    configureBinder();
}
```

Настройка Binder в форме (Шаг 3)

Связывание компонентов формы

```
private void configureBinder() {
    binder.forField(number)
        .asRequired("Укажите номер договора")
        .bind(RaContract::getNumber, RaContract::setNumber);

    binder.forField(signDate)
        .asRequired("Укажите дату")
        .bind(RaContract::getSignDate, RaContract::setSignDate);

    binder.forField(company)
        .asRequired("Выберите контрагента")
        .bind(RaContract::getCompany, RaContract::setCompany);
}
```

Логика сохранения и событий (Шаг 4)

Обработка действий сохранения

```
public void setContract(RaContract contract) {
    binder.readBean(contract);
}

private void onSave(RaContract targetEntity) {
    try {
        if (binder.writeBeanIfValid(targetEntity)) {
            // Вызов сервиса Атом.Платформы
            contractService.save(targetEntity);
            Notification.show("Сохранено успешно!");
        }
    } catch (Exception e) {
        Notification.show("Ошибка сохранения: " + e.getMessage());
    }
}
```

Паттерны связывания данных (Data Binding)

Two-Way Data Binding vs Unidirectional Data Flow в Binder

Двустороннее автосвязывание (Two-Way Binding):

binder.setBean(entity);

- *Поведение-* Любое изменение текста в поле UI мгновенно меняет свойство Java-объекта в памяти.
- *Плюсы:* Меньше кода.
- *Минусы:* Модель может оказаться в невалидном состоянии еще до того, как пользователь нажал кнопку «Сохранить».

Паттерны связывания данных (Data Binding)

Явный однонаправленный поток (Unidirectional Flow):

binder.readBean(entity) – > редактор -> binder.writeBean(entity).

- *Поведение*- Модель изолирована от UI. Изменения записываются в POJO только после успешного прохождения всей цепочки конвертеров и валидаторов.
- *Рекомендация*- В Атом.Платформе следует всегда использовать readBean / writeBean для контроля целостности данных.

Интеграция с ABAC/ACL Атом.Платформы

Автоматическая настройка UI на основе прав

```
public void applySecurityContext(UserContext user, RaContract
contract) {
    boolean canEdit = securityService.hasPermission(
        user, contract, Permission.WRITE
    );

    number.setReadOnly(!canEdit);
    signDate.setReadOnly(!canEdit);
    company.setReadOnly(!canEdit);
    save.setVisible(canEdit);
}
```

Интерфейс динамически адаптируется под правила доступа (ABAC/ACL).



Специфика Атом.Платформы:

Унифицированные формы напрямую интегрированы со слоем безопасности.

Визуальное состояние полей меняется на лету на основе вычисляемых прав доступа.

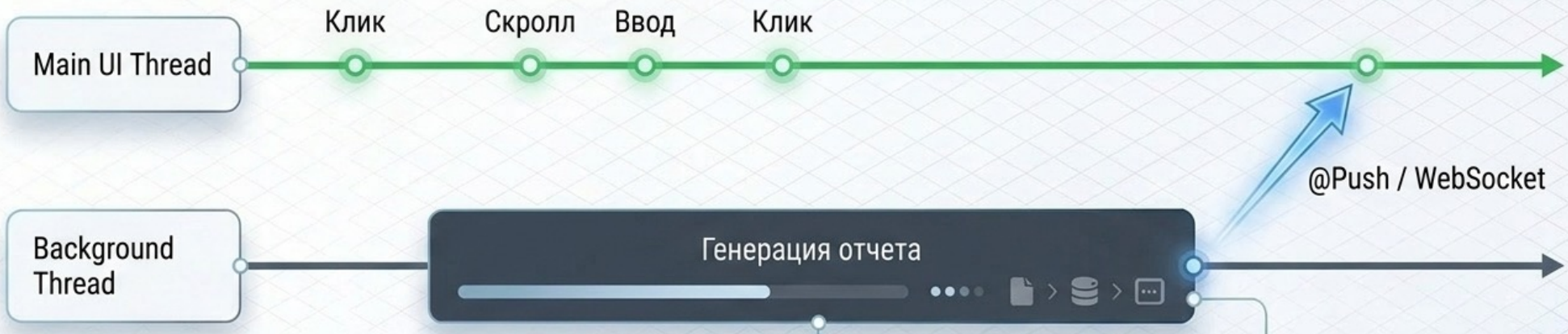
```
field.setReadOnly(!securityService.hasWriteAccess(user, entity));
```

Асинхронное обновление UI (@Push)

Асинхронные операции и Push-уведомления

- При выполнении долгих операций (генерация отчета) нельзя блокировать основной поток UI.
- Аннотация @Push включает постоянное WebSocket-соединение.
- Использование метода ui.access(() -> ...) для безопасного изменения компонентов из фонового потока.

Тяжелые фоновые задачи не должны блокировать основной поток интерфейса.



- Аннотация @Push включает постоянное двунаправленное WebSocket-соединение.
- Изменение UI из фонового потока требует синхронизации через ui.access() для предотвращения конфликтов.

```
ui.access(this::showSuccessNotification);
```

Пример использования ui.access()

Код асинхронного обновления

```
UI ui = UI.getCurrent();
CompletableFuture.runAsync(() -> {
    // Длительная операция в фоновом потоке
    byte[] pdf = reportService.generateReport(contract);

    // Возвращаемся в UI-поток для обновления компонентов
    ui.access(() -> {
        downloadButton.setEnabled(true);
        Notification.show("Отчет готов!");
    });
});
```

Сессии, память JVM и масштабирование

Жизненный цикл сессии в JVM:

- Каждый активный пользователь держит в память JVM дерево UI-компонентов (VaadinSession).
- Средний размер одной сессии: ~50–200 КБ (при правильном проектировании).
- При закрытии вкладки или таймауте сессия уничтожается, а компоненты удаляются сборщиком мусора (Garbage Collector).

Сессии, память JVM и масштабирование

Стратегии масштабирования:

- **Sticky Sessions (Закрепленные сессии)-** Балансировщик нагрузки (Nginx/HAProxy) направляет все запросы одного пользователя на один и тот же сервер приложений.
- **Очистка ресурсов (Detach listeners)-** Обязательное удаление фоновых слушателей при закрытии компонента для предотвращения утечек памяти.
- **Lazy Loading (Ленивая загрузка)-** Использование Grid с подгрузкой данных страницами из базы вместо загрузки полных списков в память сессии.

Навигация и Маршрутизация (@Route)

Маршрутизация страниц через @Route

```
@Route(value = "contracts", layout = MainLayout.class)
@PageTitle("Список договоров | Атом.Платформа")
public class ContractListView extends VerticalLayout {
    public ContractListView(ContractService service) {
        Grid<RaContract> grid = new Grid<>(RaContract.class);
        grid.setItems(service.findAll());
        add(grid);
    }
}
```

Распространенные ошибки разработчиков в Vaadin

Anti-patterns в Vaadin

- **Утечки памяти -** хранение ссылок на компоненты в синглтонах или статическом контексте.
- **Длительные вызовы в слушателях -** выполнение тяжелых SQL-запросов прямо в ClickListener без асинхронности (UI «зависает»).
- **Прямое изменение Entity без Binder -** игнорирование валидации и связывания данных.
- **Игнорирование моб. верстки -** неправильное использование FormLayout и жесткая фиксация пикселей в setWidth("1200px").

Что в итоге?

- Vaadin позволяет писать веб-интерфейсы на Java в серверно-ориентированной парадигме.
- Дерево компонентов и состояние находятся на сервере; клиент получает готовую картинку через WebSocket/XHR.
- Binder<T> — ключевой компонент для валидации, конвертации и связывания UI с моделью данных Атом.Платформы.
- Правильно спроектированный UI сочетает паттерн Master-Detail, адаптивные макеты и динамическое ограничение прав через ABAC/ACL.

Спасибо за внимание!